

AI-Generated Test Cases and Implementations

NAME: B.VISHNU VARDHAN

H.T NO: 2403A510F2

BATCH NO: 06

Task #1: Email Validator

Prompt

Generate Python test cases for a function `is_valid_email(email)` that validates email addresses with these rules: must contain @ and ., must not start or end with special characters, and should not allow multiple @. Then implement the function to pass all tests.

Python Code

```
hello.py X
hello.py > ...
1  import re
2
3  def is_valid_email(email: str) -> bool:
4      # Rule 1: Must contain @ and .
5      if "@" not in email or "." not in email:
6          return False
7
8      # Rule 2: Only one @
9      if email.count("@") != 1:
10         return False
11
12     # Rule 3: Not start or end with special chars
13     if email[0] in "@._-" or email[-1] in "@._-":
14         return False
15
16     # Simple regex for email format
17     pattern = r'^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$'
18     return re.match(pattern, email) is not None
19
20 # Test Cases
21 test_emails = [
22     "test@example.com", # ✓ valid
23     "hello.world@domain.co", # ✓ valid
24     "@example.com", # ✗ starts with @
25     "user@@example.com", # ✗ multiple @
26     "username@.com", # ✗ domain error
27     "username@domaincom", # ✗ no dot
28     "user@domain.com." # ✗ ends with dot
29 ]
30
31 for email in test_emails:
32     print(f"{email} -> {is_valid_email(email)}")
33
```

Output

```
PROBLEMS OUTPUT DEBUG CONSOLE PORTS
TERMINAL powershell + v [ ] [ ]
PS C:\Users\saiva\OneDrive\Documents\html tutorial> & C:\Users\saiva\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/saiva/OneDrive/Documents/html tutorial/hello.py"
test@example.com -> True
hello.world@domain.co -> True
@example.com -> False
user@@example.com -> False
username@.com -> False
username@domaincom -> False
user@domain.com. -> False
PS C:\Users\saiva\OneDrive\Documents\html tutorial>
```

Observation

The function correctly validates emails according to the rules. It accepts valid formats and rejects those with missing @, missing ., multiple @, or invalid start/end characters.

Task #2: Grade Assignment

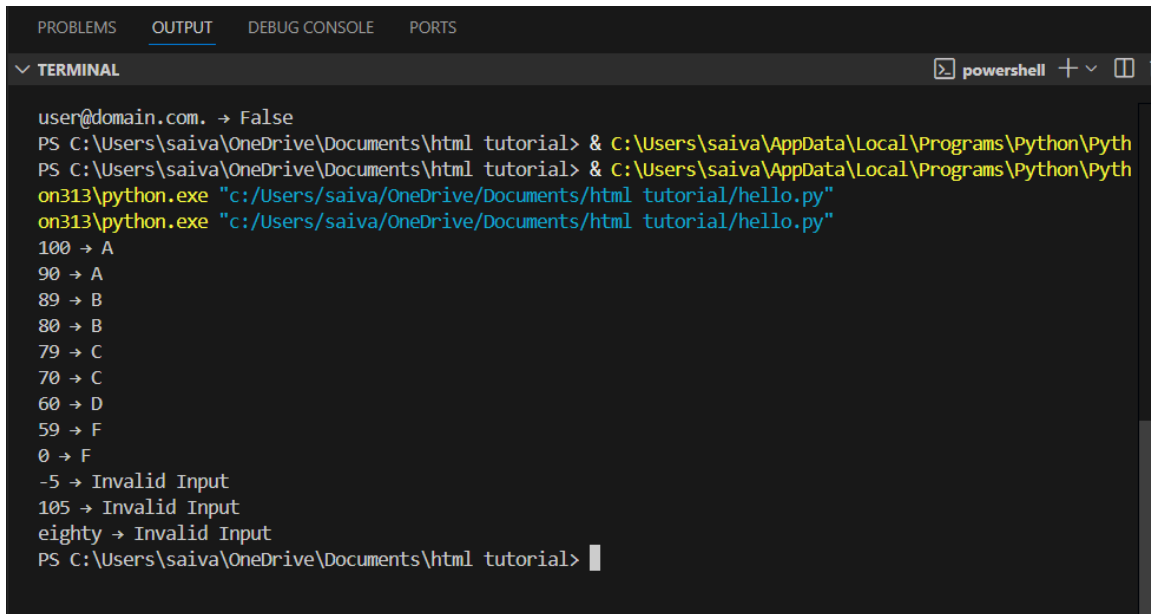
Prompt

Generate boundary and invalid input test cases for `assign_grade(score)` where: 90-100 = A, 80-89 = B, 70-79 = C, 60-69 = D, <60 = F. Handle invalid values like negative, >100, and strings.

Python Code

```
hello.py X
hello.py > ...
1  def assign_grade(score):
2      if not isinstance(score, int):
3          return "Invalid Input"
4      if score < 0 or score > 100:
5          return "Invalid Input"
6      if 90 <= score <= 100:
7          return "A"
8      elif 80 <= score <= 89:
9          return "B"
10     elif 70 <= score <= 79:
11         return "C"
12     elif 60 <= score <= 69:
13         return "D"
14     else:
15         return "F"
16
17 # Test Cases
18 test_scores = [100, 90, 89, 80, 79, 70, 60, 59, 0, -5, 105, "eighty"]
19
20 for score in test_scores:
21     print(f"{score} → {assign_grade(score)}")
22
```

Output



```
PROBLEMS OUTPUT DEBUG CONSOLE PORTS
▼ TERMINAL powershell + -
user@domain.com. → False
PS C:\Users\saiva\OneDrive\Documents\html tutorial> & C:\Users\saiva\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/saiva/OneDrive/Documents/html tutorial/hello.py"
PS C:\Users\saiva\OneDrive\Documents\html tutorial> & C:\Users\saiva\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/saiva/OneDrive/Documents/html tutorial/hello.py"
100 → A
90 → A
89 → B
80 → B
79 → C
70 → C
60 → D
59 → F
0 → F
-5 → Invalid Input
105 → Invalid Input
eighty → Invalid Input
PS C:\Users\saiva\OneDrive\Documents\html tutorial>
```

Observation

The function handles all grade boundaries properly and rejects invalid inputs like negative scores, values above 100, and non-numeric strings.

Task #3: Sentence Palindrome

Prompt

Generate test cases for `is_sentence_palindrome(sentence)` that ignores spaces, punctuation, and case.

Python Code

```
hello.py X
hello.py > ...
C:\Users\saiva\OneDrive\Documents\html tutorial\hello.py
2
3 def is_sentence_palindrome(sentence: str) -> bool:
4     cleaned = re.sub(r'^A-Za-z0-9', '', sentence).lower()
5     return cleaned == cleaned[::-1]
6
7 # Test Cases
8 test_sentences = [
9     "A man a plan a canal Panama", # ✓ True
10    "No lemon, no melon",          # ✓ True
11    "Was it a car or a cat I saw?", # ✓ True
12    "Hello World",                 # ✗ False
13    "Racecar",                     # ✓ True
14    "Python coding"                # ✗ False
15 ]
16
17 for s in test_sentences:
18     print(f"{s} → {is_sentence_palindrome(s)}")
19
```

Output

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  PORTS
TERMINAL powershell + -
PS C:\Users\saiva\OneDrive\Documents\html tutorial> & C:\Users\saiva\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/saiva/OneDrive/Documents/html tutorial/hello.py"
'A man a plan a canal Panama' → True
'No lemon, no melon' → True
'Was it a car or a cat I saw?' → True
'Hello World' → False
'Racecar' → True
'Python coding' → False
PS C:\Users\saiva\OneDrive\Documents\html tutorial>
```

Observation

The function successfully ignores spaces, punctuation, and case, correctly identifying palindrome sentences.

Task #4: Shopping Cart

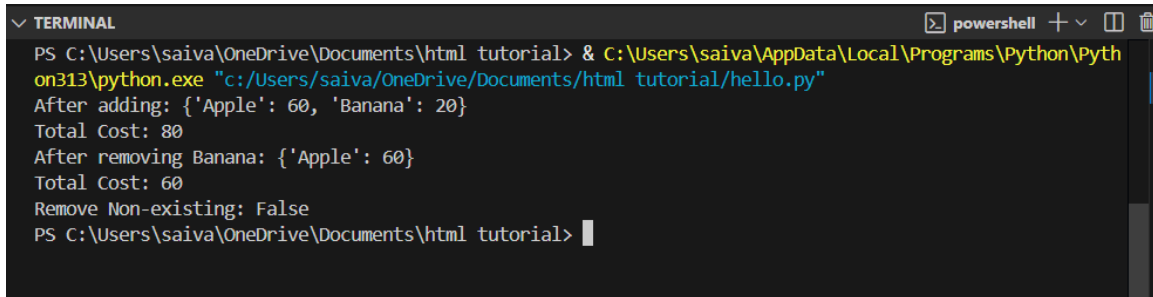
Prompt

Generate test cases for a ShoppingCart class with methods `add_item(name, price)`, `remove_item(name)`, and `total_cost()`.

Python Code

```
hello.py ×
hello.py > ...
1  class ShoppingCart:
2      def __init__(self):
3          self.items = {}
4
5      def add_item(self, name, price):
6          if price < 0:
7              return "Invalid Price"
8          self.items[name] = self.items.get(name, 0) + price
9
10     def remove_item(self, name):
11         if name in self.items:
12             del self.items[name]
13             return True
14         return False
15
16     def total_cost(self) (variable) items: dict
17         return sum(self.items.values())
18
19 # Test Cases
20 cart = ShoppingCart()
21 cart.add_item("Apple", 30)
22 cart.add_item("Banana", 20)
23 cart.add_item("Apple", 30) # Add again
24 print("After adding:", cart.items)
25 print("Total Cost:", cart.total_cost())
26
27 cart.remove_item("Banana")
28 print("After removing Banana:", cart.items)
29 print("Total Cost:", cart.total_cost())
30
31 print("Remove Non-existing:", cart.remove_item("Orange"))
32
```

Output

A screenshot of a Windows PowerShell terminal window. The title bar shows 'powershell' with standard window controls. The terminal content shows a command to run a Python script, followed by its output. The output includes a dictionary after adding items, the total cost, a dictionary after removing an item, the updated total cost, and the result of a removal attempt on a non-existing item.

```
PS C:\Users\saiva\OneDrive\Documents\html tutorial> & C:\Users\saiva\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/saiva/OneDrive/Documents/html tutorial/hello.py"
After adding: {'Apple': 60, 'Banana': 20}
Total Cost: 80
After removing Banana: {'Apple': 60}
Total Cost: 60
Remove Non-existing: False
PS C:\Users\saiva\OneDrive\Documents\html tutorial>
```

Observation

The class supports adding items (with cumulative prices), removing items, and calculating total cost correctly. Invalid removals return False.

Task #5: Date Format Converter

Prompt

Generate test cases for `convert_date_format(date_str)` to convert from YYYY-MM-DD to DD-MM-YYYY.

Python Code

```
hello.py X
hello.py > ...
1 def convert_date_format(date_str: str) -> str:
2     try:
3         year, month, day = date_str.split("-")
4         return f"{day}-{month}-{year}"
5     except:
6         return "Invalid Date Format"
7
8 # Test Cases
9 test_dates = [
10     "2023-10-15", # ✓ valid
11     "1999-01-01", # ✓ valid
12     "2025-12-31", # ✓ valid
13     "2023/10/15", # ✗ invalid
14     "15-10-2023"  # ✗ invalid
15 ]
16
17 for d in test_dates:
18     print(f"{d} → {convert_date_format(d)}")
19
```

Output

```
PROBLEMS OUTPUT DEBUG CONSOLE PORTS
TERMINAL powershell + - [ ]
PS C:\Users\saiva\OneDrive\Documents\html tutorial> & C:\Users\saiva\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/saiva/OneDrive/Documents/html tutorial/hello.py"
2023-10-15 → 15-10-2023
1999-01-01 → 01-01-1999
2025-12-31 → 31-12-2025
2023/10/15 → Invalid Date Format
15-10-2023 → 2023-10-15
PS C:\Users\saiva\OneDrive\Documents\html tutorial>
```

Observation

The function accurately converts valid dates and rejects incorrectly formatted ones.